

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 1

Student Name: A.Rohit

Roll No.: A24126552067

Date: 31-08-2026

1. TITLE

Basic JavaScript Inheritance Demonstration

2. AIM

To demonstrate the core concepts of Object-Oriented Programming (OOP) in JavaScript by implementing class structures and property inheritance using ES6 syntax.

3. OBJECTIVE

The objectives of this experiment are:

- To construct robust object-oriented blueprints using ES6 classes.
 - To implement method overriding and property inheritance between base and derived classes.
 - To execute parent and child methods on an instantiated object and display output via DOM manipulation.
-

4. THEORY

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of 'objects', which contain data and code. JavaScript supports OOP through prototype-based inheritance, which is syntactically sugar-coated by ES6 classes.

Inheritance enables code reusability by allowing a new class to take on the properties and methods of an existing class. The derived class can use methods from the parent or define its own specific behaviors.

5. ALGORITHM / PROCEDURE

1. Create an HTML framework with an empty styled container for displaying output.
 2. Define a base `Vehicle` class with a `brand` property and a `startEngine` method.
 3. Define a `Car` class that extends `Vehicle`.
 4. In the `Car` constructor, call `super()` to initialize the brand, and add a specific model property.
 5. Add a `showDetails` method inside the `Car` class.
 6. Instantiate a new `Car` object with specific brand and model arguments.
 7. Select the output container element using `getElementById`.
 8. Inject the executed results of both parent and child methods into the DOM.
-

6. PROGRAM

Source File: assignment.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Basic JS Inheritance</title>
  <style>
    body {
      font-family: Arial, sans-serif;
      padding: 20px;
      background-color: #f4f4f9;
    }
    .output-box {
      background-color: white;
      border: 2px solid #007bff;
      border-radius: 8px;
      padding: 15px;
      width: 400px;
      box-shadow: 0px 4px 8px rgba(0,0,0,0.1);
    }
    p {
      font-size: 18px;
      color: #333;
      margin: 10px 0;
    }
  </style>
</head>
<body>

  <h2>JavaScript Inheritance Demo</h2>

  <!-- This empty box is where we will print our JavaScript output -->
  <div class="output-box" id="displayArea"></div>

  <script>
    // 1. THE PARENT CLASS
    // This holds the basic properties that any vehicle would have.
    class Vehicle {
      constructor(brandName) {
        this.brand = brandName;
      }

      // A basic method all vehicles share
      startEngine() {
        return `Vroom! The ${this.brand} engine has started.`;
      }
    }

    // 2. THE CHILD CLASS
    // The 'extends' keyword means Car gets everything Vehicle has.
    class Car extends Vehicle {
      constructor(brandName, modelName) {
        // 'super()' calls the Parent class (Vehicle) constructor

```

```
        super(brandName);

        // This property is specific only to Cars
        this.model = modelName;
    }

    // A method specific to the Car class
    showDetails() {
        return `I am driving a ${this.brand} ${this.model}.`;
    }
}

// 3. CREATE AN OBJECT
// We create a new Car object.
let myCar = new Car("Toyota", "Supra");

// 4. DISPLAY ON THE WEBPAGE
// We grab the empty div and insert our text using DOM manipulation.
let display = document.getElementById("displayArea");

display.innerHTML = `
    <p><b>Method from Child:</b> ${myCar.showDetails()}</p>
    <p><b>Method from Parent:</b> ${myCar.startEngine()}</p>
`;
</script>

</body>
</html>
```

7. CODE EXPLANATION

Code / Syntax	Explanation
class Vehicle {}	Base/Parent class encapsulating standard vehicle properties.
class Car extends Vehicle	Derived/Child class inheriting from Vehicle.
super(brandName)	Invokes the Parent class constructor, passing necessary arguments.
myCar.startEngine()	Executing an inherited method derived from the Parent class.
myCar.showDetails()	Executing a distinct method exclusive to the Child class.

8. TEST CASES AND EXECUTION DATA

The experiment was executed with the following test cases to verify functionality:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Object Creation	Instantiate Car class	Object correctly initializes with brand & model	Pass
TC-02	Method Execution	Call startEngine() & showDetails()	Methods return correct formatted strings	Pass
TC-03	DOM Output	Render output strings to UI	Page displays both parent and child outputs	Pass

9. OUTPUT

Output 1

JavaScript Inheritance Demo

Method from Child: I am driving a Toyota Supra.

Method from Parent: Vroom! The Toyota engine has started.

10. RESULT

Thus, the basic JavaScript inheritance demonstration application was successfully executed and output successfully rendered to the webpage.